The Medium

This book builds a language from nothing, one piece at a time. Each piece is added when we want to say something that the pieces before it can't, and each comes with examples. Most examples are about one game, rock, paper, scissors. Piece by piece, we will write the whole game: who wins a round, how many rounds a player won, strategies that play each other, a player at the keyboard, and a score that changes as the game goes on.

0. Steps

You already know a language that works the way this one does: arithmetic.

```
(2 + 3) × 4
```

This is an **expression**: something written that says something. It says a number, though not directly. To find out which, we **work it out**, one **step** at a time:

```
(2 + 3) × 4
    becomes   5 × 4
    becomes   20
```

Each step replaces one part with something simpler that says the same: `2 + 3` and `5` are the same number. When nothing is left to work out, as in `20`, the expression is a **value**. It says what it says directly.

Notice the order. `×` can't be worked out until both its sides are numbers, so `2 + 3` goes first. A step needs values to work with, so the parts of an expression are worked out before the whole.

Our language works the same way: one step at a time, each replacing one part, until a value is left. What is different is what the parts are. Arithmetic comes with numbers, `+` and `×` built in. Our language comes with nothing built in, not even numbers. We make every kind of thing ourselves, and every way of working things out.

Writing about a language

We will also need to write about the language, not only in it. To talk about many expressions at once, every sum, say, without choosing one, we use a **placeholder**: a letter that marks a place where any expression can go.

```
e + e
```

`e` is never written in arithmetic. It is written about arithmetic. Filling each place with an expression gives a real one: `2 + 3`, or `(2 + 3) + 4`.

With placeholders, three lines describe every expression of arithmetic:

```
e ::= a number
      e + e
      e × e
```

`::=` reads "is one of", and each line is a **form**: one shape an expression can have. An expression is a number, or two expressions with `+` or `×` between them. Parentheses only group: they keep an expression in one piece. Each `e` can be any expression, made by any of the forms, so three lines describe expressions

of every size. `(2 + 3) × 4` is `e × e`, with `2 + 3` in the first place and `4` in the second.

Arithmetic also lets us write names inside it, for numbers not yet chosen:

```
square(a) = a × a
```

`a` is a **variable**: a name written in the language itself, for a value not yet chosen. Applying the function chooses it, and puts the value wherever `a` is:

```
square(3)
    becomes   3 × 3
    becomes   9
```

A placeholder and a variable both stand for something not yet chosen, but they live in different places. A placeholder, like `e`, is written about the language, and stands for any expression. A variable, like `a`, is written in the language, and stands for a value.

As we build our language, we keep a **map** of it: its forms, written with placeholders, and below them, its rules. The map grows with each section, and at the end it is the whole language. Its placeholders are single letters, and the examples never use those letters as names, so a letter on the map is always a placeholder.

Part one: saying

1. Things

Our language starts empty. First we need something to talk about. We make a kind of thing by listing every thing of that kind:

```
Hand = rock | paper | scissors
```

`|` reads "or". This **declares** a **type**, `Hand`: a hand is rock, or paper, or scissors, and nothing else. `rock`, `paper` and `scissors` are **constructors**: names that stand for themselves, and for nothing more. A type's name is written with a capital letter, and a constructor's without.

A round of the game ends in one of three ways, so here is another type:

```
Outcome = win | lose | draw
```

A constructor is the simplest expression. There is nothing to work out in `rock`, so it is also a value. Each type is declared once, and each constructor belongs to one type only: `win` is an `Outcome`, and nothing else.

On the map, `T` stands for any type's name, `c` for any constructor, `e` for any expression and `v` for any value. `| …` reads "or more like the one before, as many as needed". Each rule is tagged with what it is about:

```
T = c | …              a type, with its constructors
e ::= c                a constructor
v ::= c                a constructor

names    each type is declared once, and each constructor belongs to one type
```

So far, an expression and a value are the same thing: a constructor. That changes next.

2. Functions

Things that only stand for themselves don't do anything. We want to relate them: for each hand, the hand that beats it. That is a **function**, and we write it as **cases**: for each thing it can be given, what it gives back.

```
better = (rock) ↦ paper
       | (paper) ↦ scissors
       | (scissors) ↦ rock
```

↦ reads "gives". In each case, what is in parentheses is the **pattern**: what the case is for. What comes after ↦ is the **body**: what the case gives back. `better = …` **defines** the name `better`: from now on, it stands for this function.

`square` was written with `×`, which arithmetic already knew. Our language knows nothing yet, so `better` lists its answers, one case for each hand. Every function in our language is written this way, by cases.

To use a function, we **apply** it to an **input**, written after it in parentheses. Applying takes one step: the application **becomes** the body of the first case whose pattern the input **matches**. A constructor matches only itself.

```
better(rock)
    becomes   paper
```

`paper` is a value, so it is the **result**. Working out an expression, step after step, until no step is left, is **carrying it out**.

An input can be any expression. As in arithmetic, it is worked out before it goes in:

```
better(better(rock))
    becomes   better(paper)
    becomes   scissors
```

Comes to means "becomes, in one step or more": `better(better(rock))` comes to `scissors`.

The hand that beats the hand that beats rock is scissors, and scissors is the hand that rock beats. So a function that gives the hand each hand beats can be written with `better`:

```
worse = (rock) ↦ better(better(rock))
      | (paper) ↦ better(better(paper))
      | (scissors) ↦ better(better(scissors))

worse(paper)
    becomes   better(better(paper))
    becomes   better(scissors)
    becomes   rock
```

Its bodies are applications. Nothing in a body is worked out until the function is applied, and then only the body of the case that is used. A definition can use the names defined before it, as `worse` uses `better`.

What if a function is given something it has no case for?

```
better(win)              no case of better matches win
```

Something is left to work out, but no step applies. Carrying out is **stuck**. So carrying out can end in two ways: in a value, which is the result, or stuck.

Some things can't be written at all. Only a function can be applied, so `rock(paper)` is not an expression. And a function is used only by applying it, so `better` on its own is not an expression either.

What we write, all together, is a **program**: declarations and definitions, then one expression to carry out.

On the map, `n` stands for any defined name, and `p` for any pattern. A line marked `widened` replaces an earlier line, and allows more:

```
n = (p) ↦ e | …         a definition: n stands for a function, with its cases
p ::= c                 a constructor: matches only itself
e ::= n(e)              an application

n(v)      becomes   the body of the first case in n's definition whose pattern v matches

writing  a program is declarations and definitions, then one expression to carry out
names    each type is declared once, and each constructor belongs to one type;
         each defined name is defined once                                    widened
names    a definition can use the names defined before it
order    a step needs values: in n(e), e is worked out first
order    nothing inside a function is worked out until it is applied
ends     in a value: the result
ends     stuck: something is left to work out, but no step applies
```

3. Variables

`worse` says the same thing three times: whatever the hand, give `better` of `better` of it. To say it once, a pattern can be a **variable**: a name that matches any value, and stands, in the body, for the value it matched.

```
worse = (a) ↦ better(better(a))

worse(paper)
    becomes   better(better(paper))
    comes to  rock
```

Applying puts the input wherever the variable is, as `square(3)` put `3` wherever `a` was. This `worse` replaces the one in section 2: a program defines each name once, so when this book defines a name again, the new definition is the one used from then on.

Who wins a round? That is a question about two hands. A function can take several inputs, separated by commas, and then each case has a pattern for each:

```
result = (rock, scissors) ↦ win
       | (scissors, paper) ↦ win
       | (paper, rock) ↦ win
       | (rock, rock) ↦ draw
       | (paper, paper) ↦ draw
       | (scissors, scissors) ↦ draw
       | (a, b) ↦ lose
```

The inputs match a case when each matches its own pattern. Cases are tried in order, and the last one, all variables, matches whatever the cases before it left: it means "otherwise". Rock beats scissors, scissors beat paper, paper beats rock, and the same hands draw. Otherwise, the first player loses. Without variables, `result` would need nine cases.

```
result(paper, rock)
    becomes   win
result(rock, paper)
    becomes   lose
```

In the second, only the last case matches.

Several inputs are worked out one at a time, from left to right:

```
result(better(rock), better(paper))
    becomes   result(paper, better(paper))
    becomes   result(paper, scissors)
    becomes   lose
```

So there is never more than one next step.

The cases of one function all have the same number of patterns. That is how many inputs the function takes, and its name is always given that many: `result(rock)` is not an expression. A function can even take no inputs, and be applied to none:

```
later = () ↦ worse(rock)

later()
    becomes   worse(rock)
    comes to  scissors
```

`later` holds its body back until it is applied. Here, asking later gives the same answer as asking now. In part two, it won't.

A few rules keep every name clear. In a pattern, a name is a constructor if it was declared as one, and otherwise a variable. The variables of one case are all different: in `(a, a)`, we couldn't tell which input `a` is. None has the name of a defined name, or we couldn't tell whether it meant the function or the value matched. And a variable means something only in the body of its own case. So in a body, each name means one thing: a variable of its case, a constructor, or a defined name.

On the map, `x` stands for any variable. `p, …` means any number of patterns, even none, and likewise for the other letters:

```
n = (p, …) ↦ e | …      a definition: n stands for a function, with its cases   widened
p ::= x                 a variable: matches any value
e ::= x                 a variable
      n(e, …)           an application                                         widened

n(v, …)   becomes   the body of the first case in n's definition whose patterns
                    v, … match, with what each variable matched in its place    widened

names    the cases of one function all have the same number of patterns
names    every name is given as many as it takes: a defined name,
         as many inputs as its function takes
names    in a pattern, a name is a constructor if it was declared as one,
         and otherwise a variable
names    the variables of one case are all different,
         and none has the name of a defined name
names    a variable means something only in the body of its own case
names    in an expression, a name is a variable of the case it is in,
         or else a constructor or a defined name
order    a step needs values: in n(e, …), the inputs are worked out first,
         from left to right                                                     widened
```

4. Data

A round is two hands, one for each player. So far a value is one thing: one hand, one outcome. A constructor can **hold** things, its **contents**, of the types written after it:

```
Round = round(Hand, Hand)
```

A `Round` is `round` holding two hands. A constructor that holds things is always written with them, as many as it holds:

```
round(paper, rock)
```

This looks like an application, but it isn't one. `round` is not a function, and nothing happens: `round(paper, rock)` is already a value, a round in which the first player showed paper and the second rock. Values built by constructors are **data**. Contents are worked out before they go in, from left to right, like inputs:

```
round(better(rock), rock)
    becomes   round(paper, rock)
```

To use data, we take it apart with a pattern: the constructor again, with a pattern for each of its contents. Each can be fixed, or left open with a variable:

```
round(paper, rock)      fixed: matches only this round
round(rock, a)          the second hand left open: any round where the first showed rock
round(a, b)             both left open: any round
a                       the whole round left open
```

A value matches a pattern when it is built the same way, except where the pattern has a variable: there, any value matches, and the variable stands for it.

```
judge = (round(a, b)) ↦ result(a, b)

judge(round(paper, rock))
    becomes   result(paper, rock)
    becomes   win
```

`round(a, b)` matched `round(paper, rock)`, with `paper` as `a` and `rock` as `b`. A body can build data too. The same round, seen from the other player's side:

```
flip = (round(a, b)) ↦ round(b, a)

flip(round(paper, rock))
    becomes   round(rock, paper)
```

`Round` uses `Hand`, declared before it: a declaration can use the types declared before it. A constructor that holds nothing is written without parentheses: `rock`, never `rock()`. So a constructor on its own is data too, holding nothing, and the form `c` grows into `c(e, …)`.

On the map, `A` stands for any type:

```
T = c(A, …) | …         a type, with its constructors and the types each holds;
                        c alone if it holds nothing                              widened
A ::= T                 a type, by its name
e ::= c(e, …)           data: c with its contents                       widened from c
v ::= c(v, …)           data: c holding values                          widened from c
p ::= c(p, …)           matches data built by c, whose contents match p, …
                                                                        widened from c

names    every name is given as many as it takes: a defined name, as many inputs
         as its function takes; a constructor, as many contents as it holds,
         and none means no parentheses                                           widened
names    a declaration can use the types declared before it
order    a step needs values: the inputs of an application, and the contents
         of data, are worked out first, from left to right                       widened
```

5. Repeating

How many rounds did the first player win? To count, we need numbers, and our language has none. So we make them:

```
Nat = zero | succ(Nat)
```

`succ` reads "one more than". A `Nat` is `zero`, or one more than a `Nat`. So `zero` is a `Nat`, and so is `succ(zero)`, one, and `succ(succ(zero))`, two, and so on, without end. The type holds itself.

To add two numbers, we take the first apart, one layer at a time:

```
add = (zero, b) ↦ b
    | (succ(a), b) ↦ succ(add(a, b))
```

Zero and `b` make `b`. One more than `a`, and `b`, make one more than `a` and `b`. The second case uses `add` itself: a definition can use its own name. That is how a function goes around once for each layer, however many layers there are:

```
add(succ(succ(zero)), succ(zero))
    becomes   succ(add(succ(zero), succ(zero)))
    becomes   succ(succ(add(zero, succ(zero))))
    becomes   succ(succ(succ(zero)))
```

Each step takes a layer off the first number and puts it outside, until `zero` ends it. Two and one make three.

Once names can be used before they are defined, order no longer matters: a definition can use any defined name, even one defined after it, and a declaration any type. `worse` could just as well be defined before `better`, and `Round` declared before `Hand`.

Nothing makes a function use itself on something smaller. So carrying out can go on forever:

```
loop = (a) ↦ loop(a)

loop(rock)
    becomes   loop(rock), and so on, forever
```

Repeating needed no new form: only a name used inside its own definition, or a type inside its own declaration. Two rules widen into one, and carrying out gets a third way to end:

```
names    a declaration or a definition can use any type or defined name,
         its own included, even one declared or defined after it                widened
ends     never: carrying out goes on forever
```

6. Lists

A match is a list of rounds: no rounds, or a round followed by more rounds. The hands a player has shown are a list too, of hands instead of rounds. Only the type of the items changes, so a type can leave it open, with a variable for a type after its name:

```
List(Item) = empty | cons(Item, List(Item))
```

`Item` is a variable that stands for a type, and like a type, it is written with a capital. A `List(Round)` is `empty`, or `cons` holding a round and a `List(Round)`: `Item` is filled with `Round`. `cons` reads “followed by”, so

```
cons(round(paper, rock), cons(round(rock, rock), empty))
```

is a match of two rounds: paper against rock, then rock against rock. A type can leave more than one type open:

```
Pair(One, Two) = pair(One, Two)
```

A `Pair(Outcome, Nat)` holds an outcome and a number, as `pair(win, zero)` does. And types nest: a `List(List(Hand))` is a list of lists of hands.

`Item` means something only inside `List`’s own declaration. The variables of one declaration are all different, like `One` and `Two`, and none is named like a declared type. A type is always filled with as many types as it takes: `List` with one, `Pair` with two.

Functions need nothing like this. A function looks only at how a value is built, so a function that works through a list works through a list of anything:

```
size = (empty) ↦ zero
     | (cons(a, rest)) ↦ succ(size(rest))

size(cons(rock, cons(paper, empty)))
    comes to  succ(succ(zero))
```

Now we can count the rounds the first player won:

```
wins = (empty) ↦ zero
     | (cons(this, rest)) ↦ add(point(judge(this)), wins(rest))

point = (win) ↦ succ(zero)
      | (other) ↦ zero
```

No rounds, no wins. A round followed by the rest: a point if this round was won, added to the wins in the rest.

```
wins(cons(round(paper, rock), cons(round(rock, rock), empty)))
    comes to  add(succ(zero), add(zero, zero))
    comes to  succ(zero)
```

On the map, `X` stands for any variable for a type:

```
T(X, …) = c(A, …) | …   a type that takes types; T alone if it takes none       widened
A ::= T(A, …)           a type, filled with types; T alone if it takes none      widened
      X                 a variable for a type

names    in a declaration, every name in a type is a declared type, or one of the
         declaration's own variables; those are all different, and none is named
         like a declared type
names    every name is given as many as it takes: a defined name, as many inputs
         as its function takes; a constructor, as many contents as it holds,
         and none means no parentheses; a type, as many types as it takes       widened
```

7. Functions as values

A player picks each hand by a rule. A simple rule looks at the other player's last hand: `better` is one, "show what would have beaten it", and `worse` is another. To let rules play each other, we need to hand rules around like any other value.

So far, a defined name was always applied. It can also be written on its own. Then it stands for its function, and it is a value: nothing in it is worked out until it is applied. So a function can be an input:

```
twice = (f, a) ↦ f(f(a))

twice(better, rock)
    becomes   better(better(rock))
    comes to  scissors
```

What `twice` applies is a variable, `f`. So what is applied can now be any expression. It is worked out first, until it is a value, then the inputs, and then the application takes its step.

A rule can also be **written out**, without a name, wherever an expression can go. A player who always shows rock:

```
(a) ↦ rock
```

It is applied the same way as a named function, and can be given like any other value:

```
((a) ↦ rock)(paper)
    becomes   rock
twice((a) ↦ rock, paper)
    comes to  rock
```

The parentheses keep the function in one piece. Without them, `(paper)` would be read as part of the body, since a body reaches as far as it can. Parentheses only group: they change nothing else. And a definition adds only a name: `better` is the function `(rock) ↦ paper | (paper) ↦ scissors | (scissors) ↦ rock`, written out and named.

A function written out inside a body can use the variables around it. That is how a function makes a function from its input:

```
always = (a) ↦ (b) ↦ a

always(rock)
    becomes   (b) ↦ rock
always(rock)(paper)
    comes to  rock
```

`always(rock)` is the rule "always rock", made without a definition of its own. The input went wherever `a` was, even inside the inner function, where nothing is worked out yet. In `always(rock)(paper)`, what is applied is itself an application: it is worked out first, to the rule, and then the rule is applied.

A function can also take a rule and give back a rule:

```
twofold = (f) ↦ (a) ↦ f(f(a))

twofold(better)
    becomes   (a) ↦ better(better(a))
```

`twofold(better)` is `worse`, made from `better`.

When an inner variable has the same name as an outer one, the name means the nearer one:

```
((a) ↦ (a) ↦ a)(rock)(paper)
    becomes   ((a) ↦ a)(paper)
    becomes   paper
```

In the first step, `rock` doesn't go into the inner function: there, `a` means the inner function's own variable.

Data can hold rules too. A list of rules is a `List((Hand) → Hand)`. `(Hand) → Hand` is a **function type**, and reads "from `Hand` to `Hand`": the type of functions that, given a hand, give back a hand.

```
cons(better, cons(always(rock), empty))
```

Now rules can play each other. `play` plays a number of rounds between two rules, `f` and `g`, where `a` and `b` are the hands each player showed last. It asks each rule for its next hand, `f` answering `b` and `g` answering `a`, and `meet` puts the two hands into a round and plays the rest of the match from there:

```
play = (zero, f, g, a, b) ↦ empty
     | (succ(left), f, g, a, b) ↦ meet(left, f, g, f(b), g(a))

meet = (left, f, g, a, b) ↦ cons(round(a, b), play(left, f, g, a, b))
```

`play` and `meet` use each other. Two rounds of `better` against "always rock", both players having last shown rock:

```
play(succ(succ(zero)), better, always(rock), rock, rock)
    comes to  cons(round(paper, rock), cons(round(paper, rock), empty))
```

`better` beats "always rock" in both rounds: `wins` of this match comes to `succ(succ(zero))`.

Every value is now data or a function, and either can go anywhere a value goes:

```
(Hand) → Hand                       takes data, gives data              better
((Hand) → Hand, Hand) → Hand        takes a function, gives data        twice
(Hand) → (Hand) → Hand              takes data, gives a function        always
((Hand) → Hand) → (Hand) → Hand     takes a function, gives a function  twofold
List(Round)                         data holding data                   a match
List((Hand) → Hand)                 data holding functions              a list of rules
```

`(Hand) → (Hand) → Hand` reads "from `Hand` to `(Hand) → Hand`": like a body, what comes after → reaches as far as it can.

Now that a variable can be applied, carrying out can get stuck in two more ways:

```
twice(rock, paper)        becomes rock(rock(paper)): stuck, rock is not a function
twice(result, rock)       becomes result(result(rock)): stuck, result takes two inputs
```

Written directly, `rock(paper)` and `result(rock)` are not expressions, since the names can be seen. Through a variable, they can't be seen coming. Only carrying out finds them, and there, no step applies.

On the map, `B`, like `A`, stands for any type:

```
e ::= n                  a defined name
      (p, …) ↦ e | …     a function written out
      e(e, …)            an application                          widened from n(e, …)
v ::= n                  a defined name
      (p, …) ↦ e | …     a function written out
A ::= (A, …) → B         a function type: from A, … to B

((p, …) ↦ e | …)(v, …)
          becomes   the body of the first case whose patterns v, … match,
                    with what each variable matched put wherever its name means it
n(v, …)   becomes   the same, with the cases of n's definition                  widened

names    a variable means something in the body of its own case,
         functions written inside it included                                  widened
names    in an expression, a name means the nearest variable of that name;
         with none, it is a constructor or a defined name                      widened
names    every name is given as many as it takes: a defined name written right
         before inputs, as many inputs as its function takes; a constructor,
         as many contents as it holds, and none means no parentheses; a type,
         as many types as it takes                                             widened
order    a step needs values: in an application, what is applied is worked out
         first, then its inputs; in data, its contents; all from left to right  widened
writing  parentheses only group: (e) is e, kept in one piece
writing  a body reaches as far as it can, and so does what comes after →
```

8. What a type means

We have said that `rock` is a `Hand`, and that `better` goes from `Hand` to `Hand`. What does that mean, exactly?

A type says which values are of a kind. `rock` is a `Hand`: it is one of `Hand`'s constructors. `round(paper, rock)` is a `Round`: it is `round` holding two hands. `cons(rock, empty)` is a `List(Hand)`: it is `cons` holding a hand and a `List(Hand)`, with `Hand` in place of `Item`. So a value is of a type when it is built by one of the type's constructors, holding values of the types declared for its contents.

`round(win, lose)` can still be written and carried out. It is data, just not a `Round`. A type says what a kind is. It doesn't stop anything from being written.

`better` and `worse` are `(Hand) → Hand`: given any hand, they never get stuck, and they end in a hand. `(rock) ↦ paper` is not: given `paper`, it gets stuck. `loop` is one: given a hand, it never gets stuck, and it never ends at all. So a value is of type `(A, …) → B` when, applied to values of types `A, …`, it never gets stuck, and if it ends, it ends in a value of type `B`.

A type that holds itself is read over and over: `succ(succ(zero))` is a `Nat` because `succ(zero)` is, and `succ(zero)` is because `zero` is. A type can also hold itself in what a function gives back. A game against the computer is over, with an outcome, or it is your turn, and given your hand, it gives the rest of the

game:

```
Game = over(Outcome) | turn((Hand) → Game)
```

`turn((a) ↦ over(result(a, rock)))` is one round against rock. A game can also go on forever:

```
endless = (a) ↦ turn(endless)
```

`turn(endless)` is a `Game`. Following it never ends, but it never finds anything that isn't a `Game`. So a value is of a type that holds itself when following the definition, as far as it can be followed, never finds anything that doesn't fit.

This goes in a circle: what counts as a `Game` depends on what counts as a `Game`. For `Game`, the circle settles: the more values count as games, the more functions give back games, so the two grow together. But a type can't hold itself in what a function takes. In `Test = test((Test) → Outcome)`, the more values count as tests, the more inputs a `(Test) → Outcome` must take without getting stuck, so the fewer functions count, and the fewer values are tests. Counting more makes fewer: the circle never settles, and the definition says nothing.

Nothing in carrying out looks at types. A type is for us: it says what kind of value we mean. Whether a value is of a kind is a question we can ask about it, but asking is not a step, and this book leaves such questions aside.

```
types    a value is of type T(A, …) when it is built by one of T's constructors,
         holding values of the types declared for its contents,
         with A, … in place of T's variables
types    a value is of type (A, …) → B when, applied to values of types A, …,
         it never gets stuck, and if it ends, it ends in a value of type B
types    a type that holds itself: a value is of it when following the definition,
         as far as it can be followed, never finds anything that does not fit
types    a type can hold itself in its contents, and in what a function gives back,
         but not in what a function takes
types    nothing in carrying out looks at types
```

All of saying

That is all of saying. It has five forms of expression: variables, data, defined names, functions written out, and applications. An expression and what it comes to say the same thing, the second more directly: `worse(paper)` and `rock` are two ways of saying one hand.

Look at what can take a step. Only an application can, and only once what is applied and its inputs are values: a function meeting values. In that step, the function's patterns meet the values, and take them apart. Nothing else ever happens. Each kind of value is made one way and used one way, and every step is a use meeting what was made:

```
             made by                                 used by
data         a constructor, holding values           a pattern, which takes it apart
function     cases, each with patterns and a body    applying it to inputs
```

Part two: doing

9. Asking

So far, everything a program works with is written in it. It can't play against you, because it can't ask for your hand. An expression only says; it can't ask. So we add a way to ask:

```
Unit = unit
effect Console = read : () → Hand | show : (Outcome) → Unit
```

This declares an **effect**, `Console`: a kind of asking, with two **requests**. `read : () → Hand` reads "read takes nothing, and is answered with a hand": it asks for a hand someone types. `show : (Outcome) → Unit` takes an outcome to show on the screen, and is answered with `unit`. `unit`, the only value of `Unit`, says nothing: there is nothing to answer, only something done. Like a type's name, an effect's is written with a capital. Each effect is declared once, and each request belongs to one effect.

A request is written like an application, with as many inputs as it is declared with:

```
show(result(read(), rock))
```

This program reads your hand, and shows how it does against rock. Carrying it out, the first thing to work out is `read()`. But a request is not a value, and no step works it out. Carrying out stops there, **waiting** for an answer from outside the program:

```
show(result(read(), rock))                    waits at read()
```

Whatever runs the program asks you for a hand. Say you type `paper`. The answer goes where the request was, and carrying out goes on, until it waits again:

```
show(result(paper, rock))
    becomes   show(win)                       waits at show(win)
```

The screen shows `win`. The answer, `unit`, goes where the request was, and nothing is left to work out:

```
unit
```

The program came to a value, and on the way, it asked twice, and the world outside answered.

On the map, `F` stands for any effect's name, and `r` for any request:

```
effect F = r : (A, …) → B | …    an effect, with its requests:
                                 what each takes, and what it is answered with
e ::= r(e, …)                    a request

names    each type, effect and defined name is declared or defined once;
         each constructor belongs to one type, and each request to one effect   widened
names    every name is given as many as it takes: a defined name written right
         before inputs, as many inputs as its function takes; a constructor,
         as many contents as it holds, and none means no parentheses; a type,
         as many types as it takes; a request, as many inputs as it is
         declared with                                                          widened
names    the variables of one case are all different,
         and none has the name of a defined name or a request                   widened
names    in an expression, a name means the nearest variable of that name;
         with none, it is a constructor, a request or a defined name            widened
order    a step needs values: in an application, what is applied is worked out
         first, then its inputs; in data, its contents; in a request, its
         inputs; all from left to right                                         widened
ends     waiting: the next thing to work out is a request; when the outside
         answers, the answer goes where the request was, and carrying out goes on
ends     stuck: something is left to work out, no step applies,
         and it is not waiting                                                  widened
```

10. Answering

Sometimes the program itself should answer: to test it without a person at the keyboard, say, or to let the computer choose. A **handler** goes around an expression, and answers its requests the way the outside would:

```
handle result(read(), rock) with
      (o) ↦ o
    | (read(), k) ↦ k(paper)
```

A handler has cases, like a function. The first is for the value the expression inside comes to: here, it gives that value back. The second is for the request `read()`. In a handler's case, `read` means the request, as in a pattern a constructor's name means the constructor.

What is `k`? When the outside answered, the answer went where the request was, and carrying out went on. A handler has to be able to say that itself. So its case for a request is given what was waiting for the answer, as a function: **the rest**. Here, `result(read(), rock)` was waiting for a hand to put where `read()` is, so the rest is:

```
(y) ↦ handle result(y, rock) with …
```

Here and from now on, `with …` stands for the same handler's cases, written again. Given a hand, the rest puts it where the request was, and goes on, inside the same handler. The case for `read()` gets the rest as `k`, and `k(paper)` answers `paper`:

```
handle result(read(), rock) with …
    becomes   ((y) ↦ handle result(y, rock) with …)(paper)
    becomes   handle result(paper, rock) with …
    becomes   handle win with …
    becomes   win
```

In the first step, the next thing to work out inside the handler is `read()`, so the handler becomes the body of its case for `read()`, with the rest in place of `k`. In the third, a step happens inside the handler, as any step would. In the last, the expression inside is a value, so the handler becomes the body of its case for values.

`result(read(), rock)` never says where the hand comes from: the handler does. Another handler, or the outside, could answer differently, without changing `result(read(), rock)` at all.

A handler that has no case for a request lets it pass, to a handler further out:

```
handle (handle result(read(), rock) with (o) ↦ point(o)) with
      (o) ↦ o
    | (read(), k) ↦ k(paper)

    comes to  succ(zero)
```

The inner handler has no case for `read()`, so the outer one answers. The rest goes on inside both handlers, so the inner one still gets the outcome, `win`, and turns it into a point. Past every handler, a request goes to the outside. So the outside is like one more handler, around the whole program, that always goes on once with its answer.

Doing has two forms of expression: requests and handlers. They too are made one way and used one way: a request is made by asking, and used by a handler, which answers it. Every step is still a use meeting what was made.

On the map:

```
e ::= handle e with (p) ↦ e | (r(p, …), x) ↦ e | …
                        a handler around e: cases for the value it comes to,
                        and for requests, each with a variable for the rest

handle v with …
          becomes   the body of the first case for values that v matches,
                    with what each variable matched put wherever its name means it
handle e with …
          becomes   the body of the first case for r whose patterns v, … match,
                    with what they matched in place, and the rest in place of x,
                    when the next thing to work out in e is a request r(v, …),
                    and no handler in e around that request has a case for r

rest     the rest is (x) ↦ handle e′ with …, where e′ is e with x where the
         request was, and x is a variable used nowhere in e or in the handler
order    in a handler, steps happen in the expression inside it
ends     waiting: the next thing to work out is a request that no handler around
         it has a case for; when the outside answers, the answer goes where the
         request was, and carrying out goes on                                widened
```

11. Ways to go on

A handler holds the rest as a value, `k`, and can do with it whatever can be done with a function. Above, it went on once.

It can never go on. A player who has run out of prepared hands forfeits:

```
effect Forfeit = forfeit : () → Hand

firstOf = (cons(a, rest)) ↦ a
        | (empty) ↦ forfeit()
```

`firstOf` gives the first of a player's prepared hands, and with none left, asks to forfeit. A handler decides what forfeiting means:

```
handle judge(round(firstOf(empty), rock)) with
      (o) ↦ o
    | (forfeit(), k) ↦ lose

    comes to  lose
```

The handler never applies `k`, so the rest never happens. The round is never judged, and the handler's answer, `lose`, stands for the whole. Had it gone on with `k(rock)`, the round would have been judged a draw. This is how errors work: something asks for what can't be given, and a handler decides what happens instead.

It can go on several times. Which of a player's hands would beat rock? A handler can answer `read()` with every hand in turn, and count the wins:

```
handle result(read(), rock) with
      (o) ↦ point(o)
    | (read(), k) ↦ add(k(rock), add(k(paper), k(scissors)))

    comes to  succ(zero)
```

Each `k(…)` goes on from the request with a different hand, and comes through the first case to a point: `k(rock)` and `k(scissors)` come to `zero`, and `k(paper)` to `succ(zero)`. One hand wins. This is search: one question, every answer tried.

It can go on later. A handler can keep the rest in data, and give it back, so that whoever holds it goes on when they want. A player can be a process of its own, giving its hands one at a time. To give a hand and then go on, we need to do one thing and then another. Since inputs are worked out from left to right, a function that gives back its second input does it:

```
then = (u, a) ↦ a

effect Give = give : (Hand) → Unit

cycle = (a) ↦ then(give(a), cycle(better(a)))
```

`cycle(rock)` gives rock, then paper, then scissors, then rock again, forever. It doesn't know who takes its hands, or when. A handler can stop it at each hand, and give back the hand together with the rest:

```
Moves = done | gave(Hand, (Unit) → Moves)

moves = (task) ↦ (u) ↦ handle task() with
                          (o) ↦ done
                        | (give(a), k) ↦ gave(a, k)
```

`task` is the player, kept for later as a function with no inputs, like `later` in section 3. Given `cycle(rock)` itself, `moves` would get nothing: an input is worked out before it goes in, so `give(rock)` would be asked before any handler was around it. Kept for later, the player starts only at `task()`, inside the handler.

`moves(task)` is a function waiting for `unit`, so nothing starts until someone asks. Given `unit`, it starts the player, and stops at the first hand. The rest, `k`, waits for `unit` too, the answer to `give`. Given it, it goes on to the next hand, and stops again. With `k`, `k′` and `k″` ("k prime", "k double prime") the rests after each hand:

```
moves(() ↦ cycle(rock))(unit)      comes to  gave(rock, k)
k(unit)                            comes to  gave(paper, k′)
k′(unit)                           comes to  gave(scissors, k″)
```

`cycle` never ends, but it is carried out only as far as someone asks. This is a **generator**: it gives its items one at a time, each when it is asked for. And `Moves`, like `Game`, holds itself in what a function gives back.

So a handler can go on once, never, several times, or later. The same two forms, request and handler, say all four. None of them needed anything new on the map.

12. Memory

A score changes as the game goes on. Saying can't change anything, but it can hand a changing thing along: the score before goes in, and the score after comes out. Say we record each round: give back its outcome, and the score with a point added if the round was won:

```
record = (o, total) ↦ pair(o, add(point(o), total))
```

A function written out and applied at once can take the pair apart: its pattern names the parts.

```
((pair(o, total)) ↦ total)(pair(win, succ(zero)))
    becomes   succ(zero)
```

Now the outcomes of two rounds, with the score kept. Record the first round with the score `total`, and call what comes back `a` and `m`. Record the second with `m`, and call what comes back `b` and `t`. Give back both outcomes, with the score `t`:

```
outcomes = (first, second, total) ↦
    ((pair(a, m)) ↦
        ((pair(b, t)) ↦ pair(cons(a, cons(b, empty)), t))
        (record(judge(second), m)))
    (record(judge(first), total))

outcomes(round(paper, rock), round(rock, rock), zero)
    comes to  pair(cons(win, cons(draw, empty)), succ(zero))
```

It works, but `outcomes` doesn't care about the score: it only hands it on. So must every function that uses `outcomes`, and every function that uses those. One small idea touches everything on the way.

We want to mention the score only where it changes. With requests, we can: a function can ask for the score, and ask to set it. An effect can take variables for types, as a type can:

```
effect State(Kept) = get : () → Kept | put : (Kept) → Unit
```

In `State(Nat)`, `get` asks what the memory holds, a number, and `put` sets it. Now:

```
record = (win) ↦ then(put(succ(get())), win)
       | (o) ↦ o

outcomes = (first, second) ↦
    cons(record(judge(first)), cons(record(judge(second)), empty))
```

These replace the `record` and `outcomes` above. A won round gets the score, puts back one more, and then gives back `win`. Only `record`, where the score changes, mentions it, and `outcomes` is back to saying only what it means.

A handler answers `get` and `put`. Nothing in the language ever changes, so the handler can't keep the score in a box and change what is in it. It does what the first `outcomes` did, and hands the score along from each request to the next, but it does it in one place. Each of its cases gives back a function that waits to be told the score as it is now, `m`:

- at a value: given `m`, give back the value together with `m`;
- at `get()`: given `m`, answer `m`, and go on with the score still `m`;
- at `put(b)`: given `m`, answer `unit`, and go on with the score now `b`.

Written out:

```
keep = (task, start) ↦ (handle task() with
                          (o) ↦ ((m) ↦ pair(o, m))
                        | (get(), k) ↦ ((m) ↦ k(m)(m))
                        | (put(b), k) ↦ ((m) ↦ k(unit)(b)))(start)
```

In the case for `put(b)`, the pattern `b` matches whatever `put` was given. The rest, `k`, has the same handler around it, so it too gives back a function waiting for the score. `k(m)` answers the request with `m`, and the `(m)` after it tells the rest the score. `k(unit)(b)` answers `unit`, and tells the rest that the score is now `b`. The `(start)` at the end tells the handler the first score. And `task` is kept for later, as in `moves`.

Two rounds, the first won and the second drawn, starting from zero. `k` is the rest after the first request, `get()`, and `k′` the rest after the second, `put(succ(zero))`. Each "comes to" skips steps, and the note on the right says where it stopped:

```
keep(() ↦ outcomes(round(paper, rock), round(rock, rock)), zero)
    comes to  ((m) ↦ k(m)(m))(zero)                        at get()
    becomes   k(zero)(zero)                                get() is answered zero
    comes to  ((m) ↦ k′(unit)(succ(zero)))(zero)           at put(succ(zero))
    becomes   k′(unit)(succ(zero))                         the score is now one
    comes to  ((m) ↦ pair(cons(win, cons(draw, empty)), m))(succ(zero))
    becomes   pair(cons(win, cons(draw, empty)), succ(zero))
```

The same result as the first `outcomes`: the score went along exactly as before, but only the handler and `record` ever mention it.

With requests, the order of steps matters. In saying, the order in which inputs were worked out never changed what they came to. With a memory, it does. Put first, then get, and `get` finds the new score; get first, and it finds the old one:

```
keep(() ↦ then(put(succ(zero)), get()), zero)
    comes to  pair(succ(zero), succ(zero))
keep(() ↦ ((s) ↦ then(put(succ(zero)), s))(get()), zero)
    comes to  pair(zero, succ(zero))
```

In the second, `get()` is an input, so it is worked out first, before `put`.

`keep` never looks at the memory, so it keeps a `State(Hand)` as well as a `State(Nat)`:

```
keep(() ↦ then(put(paper), get()), rock)
    comes to  pair(paper, paper)
```

On the map:

```
effect F(X, …) = r : (A, …) → B | …    an effect that takes types;
                                       F alone if it takes none                widened
```

13. What a function may ask

A rule that reads its hand from the keyboard lets a person play: `(a) ↦ read()`. Against `better`, for one round:

```
play(succ(zero), better, (a) ↦ read(), rock, rock)
```

This asks `better` for its hand, `paper`, and then asks the person, and waits at `read()`. Say they type `scissors`. Then it comes to `cons(round(paper, scissors), empty)`: the person won.

`(a) ↦ read()` is not a `(Hand) → Hand`: applied to a hand, it doesn't end in a hand, it waits. A function type can also say what its function may ask:

```
(Hand) → Hand ! {Console}
```

reads "from `Hand` to `Hand`, and may ask `Console`". Applied to a hand, such a function may stop at `read()`, or at `show` of an outcome. With each `read()` answered with a hand, and each `show` with `unit`, it must never get stuck, and if it ends, it must end in a hand. `(a) ↦ read()` is one. "May" includes not asking, so `better` is one too. A list of rules some of which are people is a `List((Hand) → Hand ! {Console})`.

An effect that takes types is filled the same way as a type: `record` is an `(Outcome) → Outcome ! {State(Nat)}`.

So types have three forms: a type by its name, filled with types; a variable for a type; and a function type, with what it may ask.

On the map:

```
A ::= (A, …) → B ! {F(A, …), …}
                        a function type, and the effects it may ask;
                        ! {…} left out when it lists none                      widened

names    every name is given as many as it takes: a defined name written right
         before inputs, as many inputs as its function takes; a constructor,
         as many contents as it holds, and none means no parentheses; a type or
         an effect, as many types as it takes; a request, as many inputs as it
         is declared with                                                      widened
types    a value is of type (A, …) → B ! {…} when, applied to values of types
         A, …, it never gets stuck, leaves no request unanswered but those of
         the effects listed, with inputs of the types they take, and, with each
         of those answered by a value of the type it is answered with, ends,
         if it ends, in a value of type B                                      widened
```

14. The whole map

The letters: `e` an expression, `v` a value, `x` a variable, `c` a constructor, `n` a defined name, `p` a pattern, `T` a type's name, `A` and `B` types, `X` a variable for a type, `F` an effect's name, `r` a request. `e, …` means any number of them, even none, and likewise for the other letters. `| …` means more like the one before, as many as needed. "Becomes" is one step; "comes to" is one or more.

What we write

```
a program is declarations and definitions, then one expression to carry out

T(X, …) = c(A, …) | …                 a type, with its constructors and what they hold;
                                      T alone if it takes no types,
                                      c alone if it holds nothing
effect F(X, …) = r : (A, …) → B | …   an effect, with its requests: what each takes,
                                      and what it is answered with;
                                      F alone if it takes no types
n = (p, …) ↦ e | …                    a definition: n stands for a function,
                                      with its cases

parentheses only group: (e) is e, kept in one piece
a body reaches as far as it can, and so does what comes after →
```

Expressions

```
e ::= x                       a variable
      c(e, …)                 data: c with its contents
      n                       a defined name
      (p, …) ↦ e | …          a function written out
      e(e, …)                 an application
      r(e, …)                 a request
      handle e with (p) ↦ e | (r(p, …), x) ↦ e | …
                              a handler around e: cases for the value it comes to,
                              and for requests, each with a variable for the rest
```

Values and patterns

```
v ::= c(v, …)                 data: c holding values
      n                       a defined name
      (p, …) ↦ e | …          a function written out

p ::= x                       a variable: matches any value
      c(p, …)                 matches data built by c, whose contents match p, …
```

Types

```
A ::= T(A, …)                 a type, filled with types; T alone if it takes none
      X                       a variable for a type
      (A, …) → B ! {F(A, …), …}
                              a function type, and the effects it may ask;
                              ! {…} left out when it lists none
```

Steps

```
((p, …) ↦ e | …)(v, …)
          becomes   the body of the first case whose patterns v, … match,
                    with what each variable matched put wherever its name means it
n(v, …)   becomes   the same, with the cases of n's definition
handle v with …
          becomes   the body of the first case for values that v matches,
                    with what each variable matched put wherever its name means it
handle e with …
          becomes   the body of the first case for r whose patterns v, … match,
                    with what they matched in place, and the rest in place of x,
                    when the next thing to work out in e is a request r(v, …),
                    and no handler in e around that request has a case for r

the rest is (x) ↦ handle e′ with …, where e′ is e with x where the request was,
  and x is a variable used nowhere in e or in the handler
```

Where steps happen

```
a step needs values: in an application, what is applied is worked out first,
  then its inputs; in data, its contents; in a request, its inputs;
  all from left to right
in a handler, steps happen in the expression inside it
nothing inside a function is worked out until it is applied
```

How carrying out ends

```
in a value: the result
stuck: something is left to work out, no step applies, and it is not waiting
never: carrying out goes on forever
waiting: the next thing to work out is a request that no handler around it has
  a case for; when the outside answers, the answer goes where the request was,
  and carrying out goes on
```

Names

```
each type, effect and defined name is declared or defined once;
  each constructor belongs to one type, and each request to one effect
a declaration or a definition can use any type or defined name,
  its own included, even one declared or defined after it
every name is given as many as it takes: a defined name written right before
  inputs, as many inputs as its function takes; a constructor, as many contents
  as it holds, and none means no parentheses; a type or an effect, as many
  types as it takes; a request, as many inputs as it is declared with
the cases of one function all have the same number of patterns
in a declaration, every name in a type is a declared type, or one of the
  declaration's own variables; those are all different, and none is named
  like a declared type
in a pattern, a name is a constructor if it was declared as one,
  and otherwise a variable
the variables of one case are all different,
  and none has the name of a defined name or a request
a variable means something in the body of its own case,
  functions written inside it included
in an expression, a name means the nearest variable of that name;
  with none, it is a constructor, a request or a defined name
```

What a type means

```
a value is of type T(A, …) when it is built by one of T's constructors,
  holding values of the types declared for its contents,
  with A, … in place of T's variables
a value is of type (A, …) → B ! {…} when, applied to values of types A, …,
  it never gets stuck, leaves no request unanswered but those of the effects
  listed, with inputs of the types they take, and, with each of those answered
  by a value of the type it is answered with, ends, if it ends, in a value
  of type B
a type that holds itself: a value is of it when following the definition,
  as far as it can be followed, never finds anything that does not fit
a type can hold itself in its contents, and in what a function gives back,
  but not in what a function takes
nothing in carrying out looks at types
```

The shape

	made by	used by
data	a constructor, holding values	a pattern, which takes it apart
function	cases, each with patterns and a body	applying it to inputs
request	asking	a handler, which answers it

```
every step is a use meeting what was made
```

Five forms of expression for saying, two for doing, and three forms of type. Everything else is a combination of them.